

Sincronização de processos

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Sistemas Operacionais
Prof. Fabrício Sérgio de Paula

Tópicos

- Conceitos
- Problema da Seção Crítica
- Hardware de sincronização
- Semáforos
- Problemas clássicos de sincronização
- Monitores
- Exemplos de SOs

Conceitos

- Processos cooperativos
 - Compartilham dados
 - Podem afetar a execução de outros processos
- Acesso concorrente a dados compartilhados
 - Pode resultar em inconsistência: problema
 - Deve haver mecanismos para garantir a consistência

Conceitos

- Problema do produtor-consumidor
 - Buffer circular com n posições
 - Só $n-1$ posições são usadas
 - Melhoria: usar um contador (variável **counter** compartilhada)
 - Permite usar as n posições

Conceitos

- Dados compartilhados:

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int counter = 0;
```

Conceitos

- Produtor:

```
item nextProduced;
while (1) {
    while (counter == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

Conceitos

- Consumidor:

```
item nextConsumed;  
while (1) {  
    while (counter == 0)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
}
```

Conceitos

- Comando **counter++**: três instruções

registrador = counter

registrador = registrador + 1

counter = registrador

- Comando **counter--**:

registrador = counter

registrador = registrador - 1

counter = registrador

Conceitos

- Considere que **counter** tem valor 5 e a seguinte seqüência ordenada de execução:

produtor: **registrador1 = counter** (*registrador1 = 5*)

produtor: **registrador1 = registrador1 + 1** (*registrador1 = 6*)

consumidor: **registrador2 = counter** (*registrador2 = 5*)

consumidor: **registrador2 = registrador2 - 1** (*registrador2 = 4*)

produtor: **counter = registrador1** (*counter = 6*)

consumidor: **counter = registrador2** (*counter = 4*)

- Ao final, o valor de **counter** pode ser 4, 5 ou 6 (nesse caso, 4)
 - Depende da ordem de execução
 - Valor correto seria 5

Conceitos

- Inconsistências podem ocorrer sempre que há dados compartilhados
- Se resultado da execução depende da ordem de execução, há **condição de corrida** (*race condition*)
- Condições de corrida devem ser evitadas
 - Somente um processo, por vez, pode acessar/manipular algumas variáveis
 - Necessidade de sincronização entre processos: enquanto um acessa/manipula, demais aguardam

Problema da Seção Crítica

- Considere n processos: $P_0, P_1, \dots, P_{n-1}$
 - Cada processo possui uma **seção crítica**
 - Seção crítica: porção de código onde processo acessa/altera dados compartilhados
- **Problema da Seção Crítica** (PCS): garantir que nunca há dois processos executando sua seção crítica no mesmo instante
 - Em outras palavras, a execução de seções críticas é mutuamente exclusiva no tempo

Problema da Seção Crítica

- Solução para PCS: projetar um protocolo de cooperação entre os processos
 - Processos devem cooperar: devem “pedir permissão” para entrar na sua seção crítica
 - Assume-se que algumas operações são executadas **atomicamente**
 - Execução atômica de instrução: instrução não é interrompida no meio da execução (load, store, test)

Problema da Seção Crítica

- Tratamento de “permissões”:
 - **Seção de entrada**: pedido de permissão
 - Ocorre imediatamente antes da seção crítica
 - **Seção crítica**: acesso às variáveis compartilhadas
 - **Seção de saída**: liberação do acesso
 - Ocorre imediatamente depois da seção crítica
 - **Seção remanescente**: restante do código
 - Não diz respeito à seção crítica

Problema da Seção Crítica

- Estrutura geral do código de um processo P_i :

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (TRUE);
```

Problema da Seção Crítica

- Solução para PCS deve satisfazer três requisitos:
 1. **Exclusão mútua**: se processo P_i estiver executando em sua seção crítica, nenhum outro processo também estará
 2. **Progresso**: se nenhum processo estiver executando seção crítica e se alguns processos quiserem entrar:
 - Só os processos que não estiverem na seção remanescente podem participar da seleção de qual processo entrará na seção crítica
 - Essa seleção não pode ser adiada indefinidamente
 3. **Espera limitada**: se um processo pede permissão para entrar na seção crítica, existe um limite para o número de vezes que outros são permitidos entrar em suas seções críticas

Problema da Seção Crítica

- Soluções do PCS para dois processos:
 - Somente 2 processos concorrentes: P_0 e P_1
 - Processos podem compartilhar algumas variáveis para fins de sincronização

Problema da Seção Crítica

- “Solução” 1:
 - Variáveis compartilhadas
 - P_0 e P_1 são P_i e P_j e $j == 1 - i$
 - `int turn`, inicialmente `turn == 0` (mas poderia ser 1)
 - Se `turn == i`, então P_i pode entrar em sua seção crítica
 - Processo P_i
 - do {
 - while (`turn != i`)
 - ;
 - seção crítica
 - `turn = j`;
 - seção remanescente
 - } while (1);

Problema da Seção Crítica

- “Solução” 1 (cont.):
 - Características:
 - Só um processo entra na seção crítica por vez: quando $turn == i$
 - Após executar sua seção crítica, processo libera a execução somente para o outro processo
 - Deve haver alternância de execução das seções críticas, senão não há continuidade
 - Conclusão:
 - Garante a exclusão mútua
 - Não garante progresso
 - Enquanto um processo fica um grande tempo na seção remanescente, o outro não pode entrar novamente na seção crítica

Problema da Seção Crítica

- “Solução” 2:
 - Variáveis compartilhadas
 - boolean flag[2]: Inicialmente $\text{flag}[0] == \text{flag}[1] == \text{false}$
 - $\text{flag}[i] == \text{true} \Rightarrow P_i$ pronto para entrar em sua seção crítica
 - Processo P_i
 - do {
 - $\text{flag}[i] = \text{true};$
 - while ($\text{flag}[j]$)
 - ;
 - seção crítica
 - $\text{flag}[i] = \text{false};$
 - seção remanescente
 - } while (1);

Problema da Seção Crítica

- “Solução” 2 (cont.):
 - Características:
 - Somente um processo executa seção crítica por vez
 - Aguarda-se $\text{flag}[j] == \text{false}$
 - Não é necessária a alternância estrita entre processos
 - Sequência de execução:
 - P_0 executa $\text{flag}[0] = \text{true}$
 - P_1 executa $\text{flag}[1] = \text{true}$
 - ... nem P_0 nem P_1 conseguem continuar: seleção indefinida
 - Dessa forma
 - Garante exclusão mútua
 - Não garante progresso

Problema da Seção Crítica

- Solução de Peterson:
- Resolve PSC para dois processos: satisfaz exclusão mútua, progresso e espera limitada
 - Assume que instruções do tipo load e store são atômicas
 - Combina variáveis/aspectos das duas “soluções” anteriores

Problema da Seção Crítica

- Solução de Peterson:

```
do {
```

```
    flag[i] = TRUE;  
    turn = j;  
    while (flag[j] && turn == j);
```

critical section

```
    flag[i] = FALSE;
```

remainder section

```
} while (TRUE);
```

Problema da Seção Crítica

- Solução para n processos ($P_0, P_1, \dots, P_{n-1}$): algoritmo da padaria
 - Funcionamento do algoritmo:
 1. Ao entrar na loja, cada cliente recebe um número
 2. Cliente com o número menor é servido
 - O sistema de numeração sempre gera números em ordem crescente: 1,2,3,3,3,4,4,5,6,... (*timestamps*)
 - Não garante que dois processos não recebam número igual
 - Solicitações podem ser feitas simultaneamente
 - Em caso de empate, o processo com menor PID é atendido primeiro

Problema da Seção Crítica

- (cont.):
 - Notação: par ordenado (number, pid)
 - $(a, b) < (c, d)$ se $a < c$ ou se $a = c$ e $b < d$
 - $\max(a_0, \dots, a_{n-1}) = a_k$, tal que $a_k \geq a_i$ para $i = 0, \dots, n-1$
 - Dados Compartilhados
 - boolean choosing[n]
 - Inicialmente false
 - int number[n]
 - Inicialmente 0 (indica que processo não quer ser selecionado)

Problema da Seção Crítica

- (cont.):
 - Processo P_i

```
do {
    choosing[i] = true;
    number[i] = max(number[0], number[1], ..., number [n - 1])+1;
    choosing[i] = false;
    for (j = 0; j<n; j++)
    {
        while (choosing[j])
            ;
        while ( number[j] != 0 && ((number[j],j) < (number[i],i)) )
            ;
    }
    seção crítica
    number[i] = 0;
    seção remanescente
} while (1);
```

Problema da Seção Crítica

- (cont.):
 - Exclusão mútua
 - Se processo P_i está em sua seção crítica e P_k já escolheu seu número (diferente de zero), então $(number[i], i) < (number[k], k)$
 - Nesse caso, nenhum outro processo consegue ultrapassar o segundo comando while
 - Progresso
 - Processos que estão na seção remanescente possuem $number[i]=0$
 - Não participam da escolha
 - A seleção não leva um tempo indefinido
 - Espera limitada
 - Processos são servidos da forma FCFS
 - Se houver empates, com o passar do tempo, o processo que tem PID maior terá o number menor e, então, ganhará

Hardware de sincronização

- Arquiteturas provêm suporte de hardware para execução de seção crítica
 - Soluções de software (ex.: de Peterson) não possuem garantia de funcionar nas arquiteturas atuais
 - Qualquer solução para o PSC requer ferramenta simples: **lock**
 - Condições de corrida são evitadas: seções críticas protegidas por locks
 - Processo deve obter lock para entrar na seção crítica
 - Após sair da seção crítica, lock é liberado

Hardware de sincronização

- Uso de lock para acesso à seção crítica:

```
do {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
} while (TRUE);
```

Hardware de sincronização

- Suporte de hardware para monoprocessoadores: desabilitar interrupções é uma possibilidade
 - Processo corrente executa sem preempção
 - Abordagem adotada por kernels não preemptivos
 - Mecanismo muito ineficiente para multiprocessadores
 - Requer que processador atual avise demais: não é escalável
 - Outro problema: como atualizar relógio se interrupções são desabilitadas ?

Hardware de sincronização

- Arquiteturas modernas provêem instruções de hardware atômicas
 - Atômica = ininterruptível
 - Dois tipos de instruções especiais: TestAndSet() e Swap()
 - TestAndSet(): permite testar e alterar o valor de uma variável
 - Swap(): trocar o valor de duas variáveis

Hardware de sincronização

- Definição de TestAndSet():

```
boolean TestAndSet(boolean *target) {  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;  
}
```

Hardware de sincronização

- Exclusão mútua com TestAndSet():
 - Variável compartilhada: lock, inicialmente false

```
do {  
    while (TestAndSet(&lock))  
        ; // do nothing  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
} while (TRUE);
```

Hardware de sincronização

- Definição de Swap():

```
void Swap(boolean *a, boolean *b) {  
    boolean temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

Hardware de sincronização

- Exclusão mútua com Swap():
 - Variável compartilhada: lock, inicialmente false

```
do {  
    key = TRUE;  
    while (key == TRUE)  
        Swap(&lock, &key);  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
} while (TRUE);
```

Hardware de sincronização

- Algoritmos anteriores com TestAndSet() e Swap():
 - Garantem a exclusão mútua
 - Atomicidade das instruções
 - Garantem o Progresso
 - Processos na seção remanescente não interferem na seleção
 - Seleção ocorre em tempo definido
 - Não garantem a espera limitada
 - Ex.: sempre que processo P ganha CPU, tem outro dentro da seção crítica e passa todo seu tempo realizando TestAndSet
 - ou `key == true` sempre, no caso do uso da instrução Swap

Hardware de sincronização

- Solução do problema da seção crítica com TestAndSet():
 - Dados compartilhados:
 - `boolean waiting[n]; // inicialmente false`
 - `boolean lock; // inicialmente false`

Hardware de sincronização

```
do {
    waiting[i] = TRUE;
    key = TRUE;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = FALSE;

    // critical section

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = FALSE;
    else
        waiting[j] = FALSE;

    // remainder section
} while (TRUE);
```

Semáforos

- **Semáforo**: mecanismo genérico de sincronização
 - Não requer espera ocupada
 - Mais simples de usar que soluções de hardware vistas
- Um semáforo S é uma variável inteira
 - Exceto de sua inicialização, é acessada somente através de duas operações atômicas padrão:
 - `wait()`: testar e decrementar
 - `signal()`: incrementar

Semáforos

- Idéia do funcionamento de wait() e signal():

```
wait (S){  
    while (S ≤ 0)  
        ;  
    S--;  
}
```

```
signal (S){  
    S++;  
}
```

Semáforos

- Tipos de semáforos:
 - Semáforo de contagem: valor inteiro varia em um domínio “irrestrito”
 - Pode ser usado para acessar um recurso que possui número finito de instâncias
 - Semáforo é inicializado com número de instâncias do recurso
 - Semáforo binário: valor inteiro é 0 ou 1
 - Também é conhecido como **mutex lock**
 - Semáforos binários podem ser usados para implementar semáforos de contagem

Semáforos

- Uso de semáforo binário para a exclusão mútua com n processos:
 - Variável compartilhada mutex deve ser inicializada com 1

```
do {  
    wait(mutex);  
  
    // critical section  
  
    signal(mutex);  
  
    // remainder section  
} while (TRUE);
```

Semáforos

- Semáforos podem ser usado para resolver problemas diversos de sincronização, como por exemplo:
 - Processos concorrentes P_1 e P_2
 - P_1 possui comando S_1
 - P_2 possui comando S_2
 - Restrição: S_2 só pode ser executado após S_1 ser terminado
 - Implementação:
 - Processos compartilham semáforo synch, inicializado com 0
 - Processo P1:
 S_1 ;
signal(synch);
 - Processo P2:
wait(synch);
 S_2 ;

Semáforos

- Implementação de semáforos:
 - Deve garantir que dois processos não executem `wait()` e `signal()` no mesmo semáforo ao mesmo tempo
 - Então, implementação também se torna um problema da seção crítica onde o código de `wait()` e `signal()` são seções críticas
- Uso de `TestAndSet()` e `Swap()` nas implementações: **espera ocupada**
 - Desperdício de CPU
 - Útil apenas quando *locks* se mantêm por tempo curto: nem sempre é verdade
 - Espera ocupada é feita somente em porções pequenas de código

Semáforos

- Implementação de semáforos (cont.):
 - A cada semáforo é associado um valor (tipo inteiro) e uma lista de espera
 - Há também duas operações utilizadas:
 - `block()`: coloca o processo que invocou a operação na lista de espera do semáforo correspondente
 - `wakeup()`: remove um processo da lista de espera e o insere na fila de prontos

Semáforos

- Implementação de semáforos (cont.):

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Semáforos

- Implementação de semáforos (cont.):

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Semáforos

- Problemas com o uso de semáforos:
 - **Deadlock** (impasse): situação onde dois ou mais processos esperam indefinidamente por um evento que pode ser produzido apenas por um dos processos em espera
 - Ex.: Considere S e Q dois semáforos inicializados com 1:

P_0

```
wait (S);  
wait (Q);  
.  
.  
.  
signal (S);  
signal (Q);
```

P_1

```
wait (Q);  
wait (S);  
.  
.  
.  
signal (Q);  
signal (S);
```

Semáforos

- Problemas com o uso de semáforos (cont.):
 - **Starvation** (inanição): ocorre quando processo aguarda bloqueado indefinidamente dentro de um semáforo
 - Ex.: isso pode ocorrer quando remoção na lista de espera é feita da forma LIFO (primeiro processo pode nunca sair)
 - **Inversão de prioridade**: ocorre quando um processo com baixa prioridade mantém lock que é requisitado por processo de alta prioridade
 - Problema fica pior se processo com baixa prioridade é preemptado por outros mais prioritários
 - Ex.: robô Sojourner, Mars Pathfinder, 1997
 - Solução: protocolo de herança de prioridade

Problemas clássicos de sincronização

- Problemas clássicos de sincronização: soluções podem ser derivadas para resolver outros problemas
- Problema do buffer limitado: produtor-consumidor
 - Buffer: vetor de tamanho n , capaz de manter n itens
 - Solução usando semáforos
 - semáforo **mutex**: provê exclusão mútua
 - Inicialmente, vale 1
 - semáforos **empty** e **full**: contam o número de posições vazias e cheias
 - Inicialmente valem n e 0

Problemas clássicos de sincronização

- Solução para o problema do buffer limitado:

```
do {  
    . . .  
    // produce an item in nextp  
    . . .  
    wait(empty);  
    wait(mutex);  
    . . .  
    // add nextp to buffer  
    . . .  
    signal(mutex);  
    signal(full);  
} while (TRUE);
```

The structure of the producer process.

```
do {  
    wait(full);  
    wait(mutex);  
    . . .  
    // remove an item from buffer to nextc  
    . . .  
    signal(mutex);  
    signal(empty);  
    . . .  
    // consume the item in nextc  
    . . .  
} while (TRUE);
```

The structure of the consumer process.

Problemas clássicos de sincronização

- Problema dos leitores-gravadores:
 - Objeto compartilhado entre processos concorrentes
 - Alguns processos só necessitam ler: leitores
 - Outros processos necessitam ler/gravar: gravadores
 - Leitores podem acessar objeto concorrentemente
 - Nenhum problema de consistência
 - Se um gravador está acessando objeto
 - Nenhum outro processo (leitor/gravador) pode acessar

Problemas clássicos de sincronização

- Problema dos leitores-gravadores (cont.):
 - Variações do problema:
 - Variação 1: Nenhum leitor deverá esperar que outros leitores terminem, mesmo se o gravador está esperando
 - Leitores são mais privilegiados
 - Gravadores podem entrar em inanição
 - Variação 2: Se um gravador estiver esperando para acessar objeto, nenhum novo leitor poderá acessar
 - Gravadores têm maior prioridade
 - Leitores podem entrar em inanição

Problemas clássicos de sincronização

- Problema dos leitores-gravadores (cont.):
 - Implementação da variação 1:
 - Dados compartilhados:
 - semaphore **mutex** = 1, **wrt** = 1;
 - int **readcount** = 0;
 - **wrt** é comum aos leitores e gravadores:
 - Usado para garantir a exclusão mútua: impedir gravação simultaneamente com outra operação (leitura ou gravação)
 - Usado pelo primeiro e último leitor ao entrar/sair da seção crítica
 - **mutex** garante exclusão mútua quando **readcount** é atualizado
 - **readcount** controla processos que estiverem lendo

Problemas clássicos de sincronização

- Implementação da variação 1

```
do {  
    wait(mutex);  
    readcount++;  
    if (readcount == 1)  
        wait(wrt);  
    signal(mutex);  
    . . .  
    // reading is performed  
    . . .  
    wait(mutex);  
    readcount--;  
    if (readcount == 0)  
        signal(wrt);  
    signal(mutex);  
} while (TRUE);
```

The structure of a reader process.

```
do {  
    wait(wrt);  
    . . .  
    // writing is performed  
    . . .  
    signal(wrt);  
} while (TRUE);
```

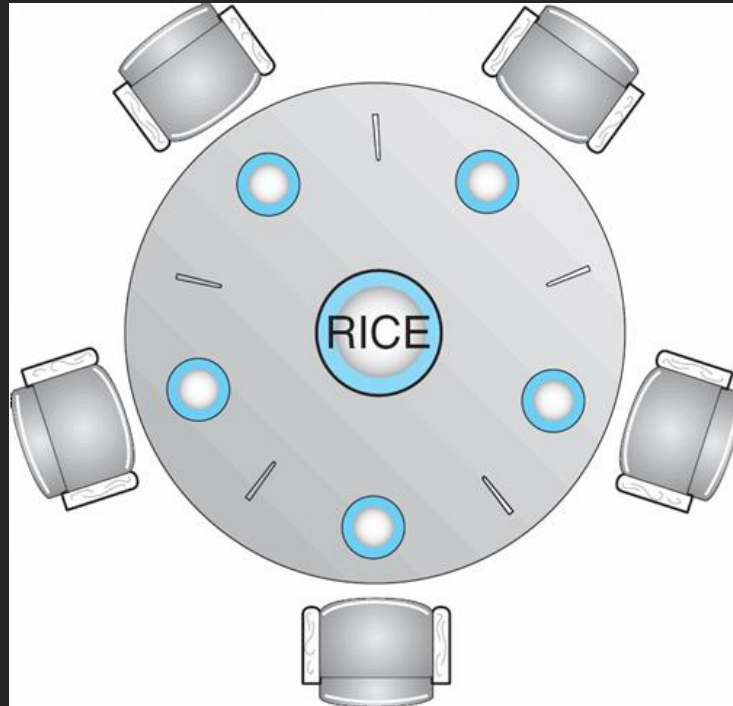
The structure of a writer process.

Problemas clássicos de sincronização

- Problema dos filósofos comensais: relacionado com diversos problemas de controle de concorrência
 - Tem grande potencial de deadlock e inanição
 - Definição do problema:
 - 5 filósofos compartilham mesa circular
 - Ficam pensando e não interagem com os colegas mas, tempo em tempo, sentem fome
 - Cada filósofo possui um prato e um talher à sua esquerda e outro à sua direita
 - Para comer, um filósofo necessita de pegar os dois talheres

Problemas clássicos de sincronização

- Problema dos filósofos comensais (cont.):



Problemas clássicos de sincronização

- Problema dos filósofos comensais (cont.):
 - “Solução”:
 - Cada talher é representado por um semáforo
 - Um filósofo tenta pegar um talher executando `wait()`
 - Libera seus talheres através de `signal()`
 - Dados compartilhados:
 - `semaphore talher[5];`

Problemas clássicos de sincronização

- Implementação da “solução” do problema dos filósofos comensais (cont.):

```
do {  
    wait(chopstick[i]);  
    wait(chopstick[(i+1) % 5]);  
  
    . . .  
    // eat  
  
    . . .  
    signal(chopstick[i]);  
    signal(chopstick[(i+1) % 5]);  
  
    . . .  
    // think  
  
    . . .  
} while (TRUE);
```

The structure of philosopher *i*.

Problemas clássicos de sincronização

- Problema dos filósofos comensais (cont.):
 - “Solução” apresentada:
 - Garante que dois vizinhos não comem simultaneamente
 - Tem potencial de ocorrer deadlock
 - Formas de resolver esse deadlock
 - Permitir, no máximo, 4 filósofos à mesa simultaneamente
 - Permitir que um filósofo só pegue os talheres se os dois estiverem disponíveis
 - Solução assimétrica
 - Filósofo ímpar pega talher à esquerda primeiro, depois à direita
 - Filósofo par pega talher à direita primeiro, depois à esquerda
 - Eliminar deadlock não elimina inanição

Monitores

- Problemas com semáforos: surgem do uso incorreto das operações
 - Ex.:
 - `signal (mutex) .... wait (mutex)`
 - `wait (mutex) ... wait (mutex)`
 - Omissão de `wait (mutex)` ou `signal (mutex)` (ou ambos)

Monitores

- Monitor: abstração de alto-nível que provê mecanismo conveniente e efetivo para sincronização de processos
 - Suporte de linguagens de programação: Concurrent Pascal, Mesa, C#, Java
- Monitor: tipo abstrato de dados = dados + operações
 - Operações: definidas pelo programador
 - São executadas com exclusão mútua
 - Podem acessar somente variáveis locais do monitor
- Monitores podem ser implementados usando semáforos

Monitores

- Sintaxe de um monitor:

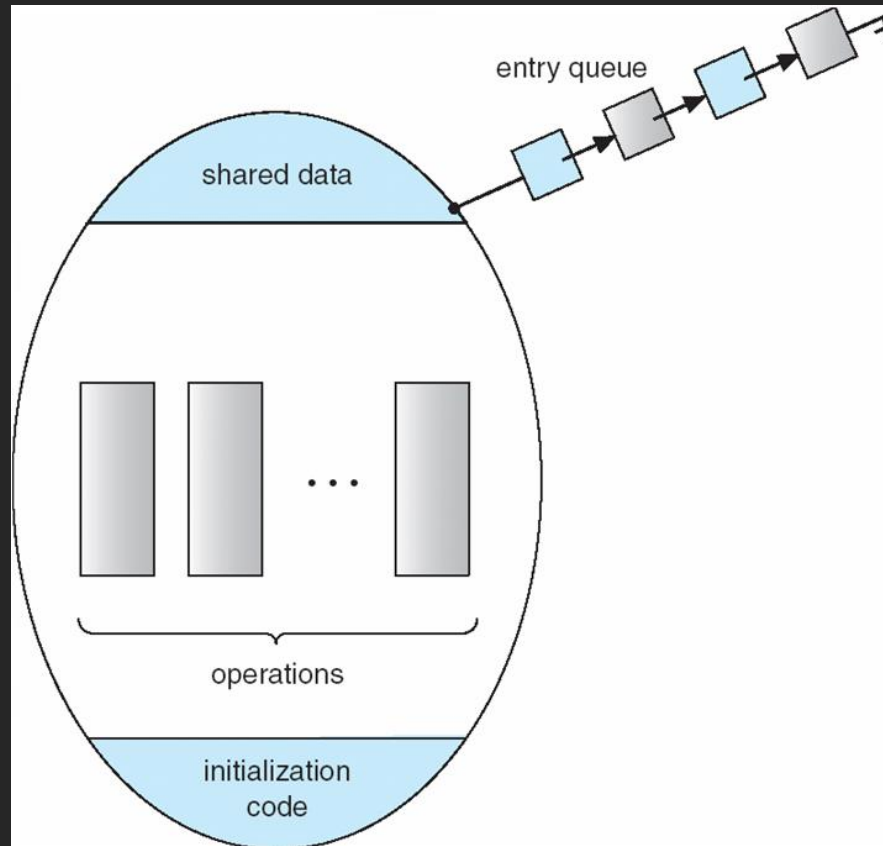
```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { .... }
    ...

    procedure Pn (...) {.....}

    Initialization code ( ....) { ... }
    ...
}
}
```

Monitores

- Visão esquematizada de um monitor:

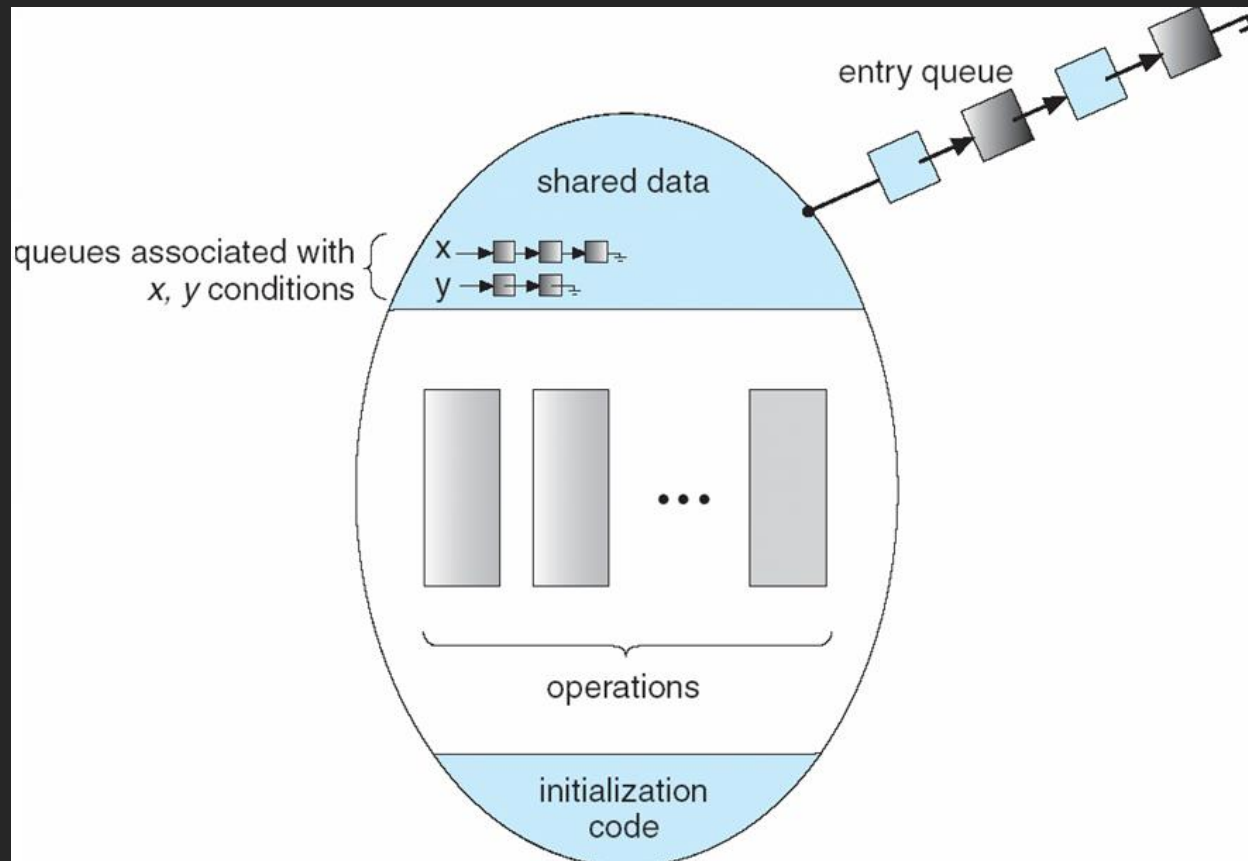


Monitores

- Monitor sozinho não é suficiente para modelar alguns esquemas de sincronização
 - Variáveis condicionais (tipo condição): as duas únicas operações são wait() e signal
 - Ex.:
 - condition x, y;
 - x.wait(); // processo é suspenso até outro executar x.signal()
 - x.signal(); // continua um processo suspenso
 - Se nenhum processo está suspenso, não possui efeito, ao contrário do signal() de um semáforo

Monitores

- Visão de um monitor com variáveis do tipo condition:



Monitores

- Solução para o Problema dos Filósofos Comensais usando monitores: livre de deadlocks
 - Filósofo só pega talheres se ambos estiverem livres
 - Estado dos filósofos: THINKING, HUNGRY, EATING
 - Variáveis condicionais:
 - `condition self[5];`
 - Cada filósofo *i* pode se bloquear quando está com fome mas não consegue obter ambos talheres
 - Operações:
 - `pickup()`: executada antes de iniciar de comer
 - `putdown()`: executada após comer

Monitores

- Solução para o problema usando um monitor:

```
monitor dp
{
    enum {THINKING, HUNGRY, EATING} state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);
    }

    void test(int i) {
        if ((state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING)) {
            state[i] = EATING;
            self[i].signal();
        }
    }

    initialization_code() {
        for (int i = 0; i < 5; i++)
            state[i] = THINKING;
    }
}
```

Exemplos de SOs

- Solaris: implementa locks com suporte multitarefa, multithread e multiprocessador
 - Mutex adaptativos: mais eficiente para códigos curtos
 - Se lock é mantido por outro processo em execução (multiprocessador) funciona como spinlock
 - Senão, thread é bloqueada
 - Variáveis condicionais e locks leitores-escretores: seções longas de código
 - Leitores-escretores: proteção de dados acessados, em geral, somente para leitura
 - Turnstile: fila de threads bloqueadas por lock

Exemplos de SOs

- Windows XP:
 - Usa mascaramento de interrupções para proteger acesso a recursos em sistemas monoprocessados
 - Usa spinlocks em sistemas multiprocessados
 - Provê dispatcher objects: atuam como mutexes ou semáforos
 - Também fornecem eventos: evento é similar a uma variável condicional

Exemplos de SOs

- Linux:
 - Antes do kernel 2.6: desabilita interrupções para seções críticas curtas
 - Versão 2.6 e posterior: totalmente preemptivo
 - Ferramentas de sincronização: semáforos e spinlocks

Exemplos de SOs

- Pthreads: mutexes, semáforos, variáveis condicionais e read-write locks
 - API não é parte de um kernel em particular
 - Algumas extensões da API implementam outras facilidades (ex.: spinlocks), o que não é totalmente portátil de uma implementação para outra